

APPLICATION OVERVIEW DOCUMENT

ClassX

Congratulations on completing your application definition! This document presents a summary overview of your application as defined in the Pega platform. It encapsulates the key elements of your application's design, highlighting its core functionalities and overall structure.

Please note that this is a high-level view and does not include granular details like data models or sample data. It's an ideal resource for a quick reference, facilitating discussions among team members and stakeholders, and for guiding strategic decisions in your application development process.

Blueprint ID: BP-2265103

ClassX — AI-Powered Academic Management Platform ## 1. Project Overview **ClassX** is a secure, AI-powered academic management platform designed for **engineering colleges and high schools**. The platform connects **HODs, teachers, and students** in one centralized system and aims to automate academic activities such as assignments, assessments, AI-assisted evaluation, attendance, timetables, notifications, academic analytics, and student rewards. The main objective of ClassX is **maximum academic automation with fair, consistent, evidence-based evaluation**, while reducing repetitive work for teachers and administrative workload for HODs. The platform will be designed for **multiple colleges and thousands of users**, with strict separation of data between colleges, departments, classes, subjects, teachers, and students. --- # 2. Main Objectives ClassX is built around these objectives: * Reduce manual academic work. * Automate routine academic processes. * Provide consistent and transparent evaluation. * Reduce teacher bias/favoritism in marking. * Detect possible copied/similar assignment answers. * Allow students to submit assignments digitally. * Allow teachers to conduct MCQ assessments directly inside the application. * Automatically calculate marks and academic performance. * Maintain attendance digitally. * Provide HOD-level academic monitoring. * Connect HOD, teacher, and student data. * Provide automatic notifications for important academic events. * Provide rewards after students complete assigned academic work. * Support multiple colleges on the same platform while keeping their data isolated. * Handle large numbers of users without application crashes. * Preserve academic records at the end of each academic/semester period while reducing unnecessary server storage. --- # 3. Target Users ClassX has three primary user roles: ### 1. HOD The HOD has the highest academic control within the department. ### 2. Teacher Teachers manage only the classes and subjects assigned to them. ### 3. Student Students access only the classes, subjects, assignments, assessments, attendance, timetable, marks, and other information assigned to them. --- # 4. College-Level Structure ClassX will support multiple institutions. The system hierarchy will be: ``text University ↓ College ↓ College Code ↓ Education Type (Degree / Diploma) ↓ Program ↓ Department ↓ Academic Year / Group ↓ Semester ↓ Subjects ↓ Teachers + Students `` ## College Identification If two colleges have the same or similar names, the platform will distinguish them using a **unique College Code**. The HOD registration process will therefore include: * College name * College code * University * Education type * Program * Department * Name * Email * Password This prevents different colleges from accidentally sharing academic data. --- # 5. Account Creation Every user will create an account using: * Email * Password * Name * Role ## HOD HOD information: * Name * Email * Password * University * College * College Code * Education type * Program * Department No student enrollment number is required for HOD. ## Teacher Teacher information: * Name * Email * Password * Department The HOD will assign teachers to specific subjects/classes. Teacher assignment can be performed through the teacher's registered email. ## Student Students will provide: * Name * Email * Password * Enrollment number * Roll number * Department * Academic information Students will join the appropriate academic structure through an **HOD-generated secure joining link**. --- # 6. HOD Registration and Department Setup After logging in, the HOD will create/manage the department's academic structure. For example: ``text AIML Department FY |—— Subject 1 |—— Subject 2 |—— Subject 3 |—— Subject 4 |—— Subject 5 SY |—— Subject 1 |—— Subject 2 |—— Subject 3 |—— Subject 4 |—— Subject 5 TY |—— Subject 1 |—— Subject 2 |—— Subject 3 |—— Subject 4 |—— Subject 5 Final Year |—— Subject 1 |—— Subject 2 |—— Subject 3 |—— Subject 4 |—— Subject 5 `` The HOD will decide which subjects belong to each academic group/semester. --- # 7. Mandatory and Optional Subjects Some programs have optional subjects. Therefore, when a student joins a department: ### Mandatory subjects The student is automatically enrolled. ### Optional subjects The student will see available options and manually select the subject(s) they want. This prevents unnecessary subject enrollment. --- # 8. Student Academic-Year / Class Management The HOD will maintain the student list. If a student: * fails a semester, * repeats a class, * changes academic status, * or needs to remain in a previous academic group, the HOD can update the student's class/group information. A student can therefore join the correct academic class based on the updated academic information without requiring unnecessary manual approval. --- # 9. HOD → Teacher Allocation The HOD will assign teachers to subjects/classes. Example: ``text FY AIML AI → Teacher A DBMS → Teacher B DAA → Teacher C Python → Teacher D Mathematics → Teacher E `` Teachers will only have access to the subjects/classes allocated to them. They cannot access unrelated classes or subjects. --- # 10. Student Joining System The HOD will generate a **secure joining link + security code**. The HOD can share the link with students through the appropriate college communication channel. After opening the link, the student creates/logs into their account and provides the required student information. Once successfully verified, the student is automatically connected to the appropriate academic structure. For example: ``text HOD ↓ Department Joining Link ↓ Student joins ↓ Student verification ↓ Mandatory subjects automatically added ↓ Optional subjects shown for selection `` --- # 11. Student Application The student application will have **four primary sections**. ## Page 1 — Home / Classes Students will see their assigned classrooms/subjects. For example: ``text My Classes AI DBMS DAA Python Mathematics `` Opening a subject will provide access to its relevant academic content. This can include: * Assignments * Assessments * Syllabus * Teacher notices * Results * Academic information --- # 12. Student Evaluation Page The second page will display subject-wise academic performance. Students can view: * Assignment marks * Assessment marks * Total marks * Performance * Subject-wise progress * AI-generated feedback where applicable * Overall academic evaluation The information will be connected using the student's unique academic identifier/PRN/enrollment information. --- # 13. Student Attendance Page The third page will display the student's attendance. The student can see: ### Daily Attendance ``text Date Subject Status 01 Aug AI Present 02 Aug DBMS Absent 03 Aug DAA Present `` ### Monthly Attendance The system will calculate attendance for each month. ### Selected-

Month Total Students can select a month or relevant period and view their total attendance. Students will only see **their own attendance information**. --- # 14. Student Timetable Page The fourth page will show the student's timetable. It can include: * Day * Period * Subject * Teacher * Classroom/lab * Practical batch * Lecture status If the HOD changes the timetable, the updated timetable will automatically become available to relevant users and notifications can be generated. --- # 15. Teacher Application Teachers will also have four main sections. ## Page 1 — My Classes Teachers will see only classes/subjects assigned to them. From here they can manage: * Assignments * Assessments * Syllabus PDFs * Academic materials * Deadlines * Student submissions * Feedback --- # 16. Assignment Creation Teachers can create an assignment by providing: * Title * Description * Subject * Instructions * Deadline * Maximum marks * Reference material if required Once published, the assignment becomes visible to the relevant students. Teachers can modify allowed settings such as: * Deadline * Instructions * Other permitted assignment details --- # 17. Student Assignment Submission Students will submit assignments through the application. The preferred submission method is **PDF scanning directly through the application**. The system will: 1. Scan the document. 2. Optimize/compress it. 3. Maintain readable quality. 4. Reduce unnecessary file size. 5. Upload the document securely. The PDF must remain clear enough for teachers and AI processing to read. --- # 18. Assignment Privacy A student's assignment submission will **not be visible to other students**. Example: ``text Student A ↓ Assignment A ↓ Teacher A Student B ↓ Assignment B ↓ Teacher A `` Student B cannot access Student A's submission. Similarly, unauthorized teachers/classes/colleges cannot access the submission. This is a critical security requirement. --- # 19. AI Assignment Evaluation One of ClassX's major features is AI-assisted assignment evaluation. The system will process submitted PDFs. The general pipeline will be: ``text Student PDF ↓ PDF Scanner / Processing ↓ OCR / Text Extraction ↓ Question + Answer Detection ↓ Reference Answer / Rubric ↓ AI Evaluation ↓ Marks + Explanation + Feedback ↓ Similarity / Copy Analysis ↓ Teacher Review `` --- # 20. AI-Based Marking The AI will evaluate answers using: * Question relevance * Correctness * Required concepts * Reference material * Teacher-provided rubric * Expected answer structure * Answer completeness The system should provide: ``text Question 1 Marks: 8/10 Reason: Correct concept and explanation, but one important point is missing. Confidence: High `` The purpose is to make evaluation **consistent and evidence-based**, rather than dependent entirely on teacher preference. --- # 21. Copy / Similarity Detection ClassX will compare submitted answers to detect potentially copied or highly similar content. However, the system should **not automatically declare a student guilty of copying solely because two answers are similar**. This is particularly important because: * Students in the same subject receive the same questions. * Technical answers can naturally contain similar terminology. * Standard definitions may legitimately be identical. Therefore, the AI should produce evidence such as: ``text Similarity Score: 87% Potentially matching section: Question 3 Matching concepts: ... `` The result should be treated as **similarity evidence**, not an automatic accusation. --- # 22. Cross-Class Similarity If two different classes have the same subject or same assignment, their answers can naturally be similar. Therefore, similarity detection should work across relevant assignments while understanding: * Same question * Same reference material * Standard technical terms * Common definitions * Genuine independent answers The system should distinguish **legitimate similarity from suspicious copying** as much as technically possible. --- # 23. Assessment System Teachers can conduct assessments directly inside ClassX. This removes the need for external Google Forms for routine MCQ assessments. Teachers can create: * MCQ tests * Question banks * Multiple-choice questions * Marks * Time limits * Start/end time * Number of questions * Randomized questions where required --- # 24. Assessment Attempt Students can start an assessment directly from the application. The assessment can be: * Time-bound * Question-limited * Automatically submitted when time expires Results can be automatically calculated for objective questions. --- # 25. Assessment Security / Anti-Malpractice ClassX will aim to reduce malpractice through technical controls such as: * Full-screen assessment mode * Restricted navigation * Copy/paste restrictions where technically supported * Screenshot deterrence/restrictions where supported by the platform * Time limits * Question randomization * Option randomization * Automatic submission * Attempt logs * Suspicious activity logging However, **no normal mobile/desktop app can guarantee 100% prevention of screenshots or cheating** because the user can use another device or external camera. Therefore, ClassX should aim for **strong deterrence + activity logging + assessment design**, rather than claiming impossible 100% prevention. --- # 26. Teacher Evaluation Dashboard The teacher's second section will contain student academic records in an **Excel-like table**. Example: | Roll No | Name | Assignment 1 | Assignment 2 | Assessment 1 | Assessment 2 | Total | Evaluation | |-----|-----|-----|-----|-----|-----|-----|-----| | 01 | Student A | 9 | 8 | 18 | 17 | 52 | Good | | 02 | Student B | 7 | 9 | 15 | 18 | 49 | Average | Teachers can view: * Student-wise marks * Assignment marks * Assessment marks * Total marks * Performance trends * Growth * AI evaluation * Feedback --- # 27. Teacher Attendance The teacher's third section will contain the attendance records for their allocated classes. Teachers will take attendance manually. Example: ``text Date: 07 August Roll No Student Present 01 Student A ✓ 02 Student B × 03 Student C ✓ `` Once attendance is saved, the record becomes part of the academic attendance history. --- # 28. Attendance Evaluation Attendance will be automatically evaluated. Two main views are required: ### Monthly ``text August Total Lectures: 40 Present: 35 Absent: 5 Attendance: 87.5% `` ### Selected Period Teacher/HOD can select a relevant month/period and view total attendance. The teacher can see the attendance of all students in their allocated subject/class. --- # 29. Attendance Locking After a teacher saves attendance for a lecture, the attendance record should be treated as immutable under normal operation. If correction is genuinely necessary, the system should use a controlled correction workflow rather than allowing silent editing. This provides an audit trail and reduces manipulation. --- # 30. HOD Dashboard The HOD has the highest academic access within their department. The HOD dashboard has four primary sections. --- # 31. HOD Page 1 — Academic Groups The HOD will manage groups such as: ``text FY AIML SY AIML TY AIML Final Year AIML `` Each group contains its relevant subjects/classes. Example: ``text FY AIML |—— AI |—— DBMS |—— DAA |—— Python |—— Mathematics `` --- # 32. HOD Page 2 —

Academic Data The HOD can enter a group and view the relevant class/subject data. The HOD can access: * Student information * Teacher allocation * Assignment performance * Assessment performance * Overall evaluation * Subject-wise performance

The data is connected using unique student identifiers such as PRN/enrollment number and internal roll number. --- # 33. HOD Page 3 — Attendance The HOD can see attendance records for classes under their department. The data will be presented in an Excel-like format. Example: | Roll No | Name | AI | DBMS | DAA | Python | Overall | | ----- | ----- | -: | -: | -: | -: | -: | | | 01 | Student A | 90% | 88% | 92% | 85% | 88.75% | | 02 | Student B | 78% | 82% | 75% | 80% | 78.75% | The HOD can inspect subject-wise attendance. --- # 34. HOD Page 4 — Timetable The HOD will create and manage the department timetable. The timetable system must support: * Monday-Saturday or configured working days * Period number * Subject * Teacher * Classroom * Laboratory * Practical * Batch 1 * Batch 2 * Other practical batches * Lecture cancellation * Off period * Weekly schedule --- # 35. Recurring Timetable Once a timetable is created, it remains active for subsequent weeks until the HOD updates it. For example: ``text Monday Period 1 → AI Period 2 → DBMS Period 3 → DAA Period 4 → Practical Batch 1 `` The same timetable continues next week unless modified. --- # 36. Lecture Off / Cancellation Teachers can have an **Off Period / Lecture Off** option where permitted. If a lecture is cancelled or marked off, relevant students can be notified. This prevents students from unnecessarily attending cancelled lectures. --- # 37. Notifications Any information that students need to know should generate an appropriate notification. Examples: * New assignment * Assignment deadline * Deadline change * New assessment * Assessment starting * Assessment ending * Timetable change * Lecture cancellation * New notice * Important HOD announcement * Result published Notifications should be role- and class-specific. --- # 38. HOD Notices The HOD can publish notices to: * Entire department * Academic group * Specific class * Relevant students Teachers can also publish information only to their assigned students where permitted. --- # 39. Reward System ClassX will include a student motivation system. After a student completes an assigned academic task, the system can automatically allocate a reward. The planned reward format is approximately a **15-minute audio clip / audiobook-style reward**. Example: ``text Assignment Completed ✓ ↓ System verifies completion ↓ Reward unlocked 🎧 ↓ Student listens ↓ Reward automatically clears `` Once consumed, the reward should no longer remain available as an unclaimed reward. --- # 40. Academic Performance System The platform will calculate student performance automatically from available academic data. Possible components: ``text Assignments + Assessments + Attendance + Subject performance + Academic growth `` Teachers will primarily see the performance of students in their assigned subjects. HODs will have department-level and subject-level visibility. Students will see only their own performance. --- # 41. Excel-Like Academic Sheets The platform will provide spreadsheet-style dashboards rather than forcing users to manually manage separate Excel files. Teacher view: ``text Student → Assignment → Assessment → Total → Evaluation `` HOD view: ``text Student → Subject 1 → Subject 2 → Subject 3 → ... → Overall `` The system should also allow authorized users to **export academic records to Excel/CSV/PDF** when required. --- # 42. Semester / Academic Period Closure The system is designed around academic periods. For engineering programs, an academic semester can generally be treated as a fixed academic period. At the end of the period: 1. Academic records are finalized. 2. HOD can generate/export the required records. 3. The finalized record becomes non-editable under normal access. 4. Data can be archived. 5. Active classroom data can be cleared/removed from high-cost operational storage according to the company's retention policy. This reduces unnecessary server load and storage cost. Important records should not simply be deleted without a proper **backup, archival and retention policy**. --- # 43. Data Isolation This is one of the most important ClassX requirements. Different colleges must never accidentally access each other's data. Example: ``text College A └─ AIML └─ FY └─ Students College B └─ AIML └─ FY └─ Students `` Even if both colleges have: * Same department * Same semester * Same subject * Same assignment their records remain isolated. --- # 44. Role-Based Access Control ClassX will use role-based access. ``text HOD | └─ Department management └─ Teacher allocation └─ Student allocation └─ Academic data └─ Attendance └─ Timetable └─ Notices Teacher | └─ Assigned classes └─ Assignments └─ Assessments └─ Student submissions └─ Marks └─ Attendance └─ Subject performance Student | └─ Assigned classes └─ Assignment submission └─ Assessments └─ Own marks └─ Own attendance └─ Timetable └─ Notifications `` A user should never receive access simply because a button is visible. \*\*The backend must independently enforce every permission.\*\* --- # 45. Assignment Privacy and Access Control A student submission must be accessible only to authorized users. Example: ``text Student A ↓ Assignment Submission ↓ Teacher assigned to that subject ↓ Authorized HOD `` Not accessible to: ``text Student B × Teacher from another subject × Teacher from another class × Another college × Unauthorized API request × `` --- # 46. Security Security will be a core part of the system. The platform should include: * Secure authentication * Password hashing * Role-based authorization * College-level data isolation * Class-level data isolation * Subject-level authorization * Secure file storage * Private file URLs/access control * Encryption where appropriate * API validation * Audit logs * Rate limiting * Secure session/token handling * Backup and recovery * Monitoring * Security testing --- # 47. Scalability The long-term goal is to support many colleges and a large number of students. The architecture should therefore be designed for: * Concurrent assessment attempts * Large numbers of assignment uploads * PDF processing * AI processing * Large attendance datasets * Large database tables * Multiple colleges * High traffic * Automatic scaling where appropriate The prototype target is to reliably handle approximately **1,500 student records and realistic concurrent usage** before moving toward larger deployments. --- # 48. AI Processing Architecture AI-related processing should not block the main application. A possible architecture: ``text Student Uploads PDF ↓ File Storage ↓ Background Job ↓ OCR / Text Extraction ↓ AI Evaluation ↓ Similarity Analysis ↓ Result Database ↓ Teacher Dashboard `` This prevents heavy AI/PDF processing from freezing the main application. --- # 49. Recommended Technical Architecture ### Frontend **Flutter + Dart** One codebase can support the primary mobile application experience. ### Backend **Python + FastAPI** Used for: * APIs * Authentication * Authorization * Business logic *

Assignment management * Assessment management * Attendance * Timetable * Notifications ### Database **PostgreSQL**
 Used for: * Users * Colleges * Departments * Classes * Subjects * Teachers * Students * Assignments * Assessments * Marks *
 Attendance * Timetable * Notifications ### File Storage Object storage for: * PDFs * Syllabus * Assignment documents * Other
 academic files ### AI Python-based AI service using suitable: * OCR * LLM APIs * Embedding models * Semantic similarity
 models * Evaluation/rubric pipelines ### DevOps * Git * GitHub * Docker * Cloud infrastructure * CI/CD * Monitoring * Logging
 * Load testing --- # 50. Four-Member Development Structure ### Person 1 — Frontend / UI-UX Responsible for: * Figma * UI/UX
 * Flutter * Dart * Student app * Teacher app * HOD app * API integration ### Person 2 — Backend / Database Responsible for:
 * Python * FastAPI * PostgreSQL * REST APIs * Authentication * Authorization * Database architecture * File handling * Backend
 business logic ### Person 3 — AI / ML Responsible for: * Python * OCR * NLP * LLM APIs * Embeddings * Semantic similarity *
 AI marking * Rubric-based evaluation * Copy/similarity analysis ### Person 4 — Cloud / DevOps / Security Responsible for: *
 Linux * Docker * Cloud * CI/CD * Infrastructure * Monitoring * Security * Load testing * Backup/recovery * Scalability All four
 members will learn **Python basics, Git/GitHub, API fundamentals and basic database concepts** together. --- # 51. Prototype
 Development Roadmap ## Phase 1 — Foundation Learn: * Programming * Git/GitHub * SQL * API basics * UI/UX fundamentals
 ## Phase 2 — Core System Build: * Login * HOD * Teacher * Student * College * Department * Class * Subject * User
 permissions ## Phase 3 — Assignment System Build: * Assignment creation * PDF scanning * PDF upload * Submission privacy
 * Teacher review ## Phase 4 — AI Build: * OCR * Answer extraction * AI evaluation * Marks * Feedback * Similarity analysis ##
 Phase 5 — Assessments Build: * MCQ creation * Question bank * Timer * Randomization * Automatic evaluation * Results ##
 Phase 6 — Attendance Build: * Teacher attendance * Daily attendance * Monthly calculation * Selected-period calculation *
 Student view * Teacher spreadsheet * HOD spreadsheet ## Phase 7 — Timetable Build: * Weekly timetable * Practical batches *
 Lecture cancellation * Off period * Notifications ## Phase 8 — Rewards + Notifications Build: * Audio reward * Completion
 detection * Reward unlocking * Automatic clearing after consumption * Notifications ## Phase 9 — Security Test: * Role
 permissions * College isolation * Class isolation * File access * API security * Authentication * Audit logs ## Phase 10 — Scale
 Testing Test with: * 1,500+ students * Multiple classes * Multiple teachers * Large assignment datasets * Many simultaneous
 assessments * Multiple PDF uploads * AI processing workloads ## Phase 11 — Final Prototype Deliver: * HOD application *
 Teacher application * Student application * Backend * Database * AI system * Cloud deployment * Security * Documentation *
 Demo --- # 52. Core Product Philosophy ClassX is not intended to simply replace Google Classroom or Google Forms. Its main
 differentiation is the **interconnected academic automation layer**: ``text HOD ↓ Academic Structure ↓ Teacher Allocation ↓
 Students ↓ Assignments ↓ AI Evaluation ↓ Assessments ↓ Attendance ↓ Timetable ↓ Academic Analytics ↓ Automated
 Notifications ↓ Student Rewards `` The ultimate objective is: > **Minimum manual academic administration with maximum
 transparency, automation, security and consistency.** ClassX should make the academic workflow easier for HODs, reduce
 repetitive workload for teachers, and give students a single place to manage their academic activities and monitor their own
 progress.

Organization name	Location	Industry	Language
Other	India	Other	English

Application Architecture

Users access the application through defined channels, assume specific roles, and trigger workflows that manage how work progresses. Those workflows operate on shared Data Objects to ensure the system stays consistent and up to date.

8	9	27
PERSONAS AND CHANNELS	WORKFLOWS	DATA OBJECTS
 Student	 Assignment Management	 System of record: Pega(Local) University
 Teacher	 Assessment Execution	 System of record: Pega(Local) College
 Head of Department	 Attendance Tracking	 System of record: Pega(Local) Department
 Parent	 Timetable Management	 System of record: Pega(Local) Program
 College Administrator	 Academic Performance Analytics	 System of record: Pega(Local) Academic Group
 AI Evaluator	 Reward Allocation	 System of record: Pega(Local) Semester
 Technical Support	 Notification Dispatch	 System of record: Pega(Local) Subject
 Application Control Agent	 Department Setup	 System of record: Pega(Local) Teacher
	 Semester Closure	 System of record: Pega(Local) Student
		 System of record: Pega(Local) Assignment
		 System of record: Cloud File Storage Service Assignment Submission
		 System of record: Pega(Local) Assessment
		 System of record: Pega(Local) Assessment Attempt
		 System of record: Pega(Local) Attendance Record
		 System of record: Pega(Local) Timetable
		 System of record: Notification Service Notification
		 System of record: Pega(Local) Notice

	System of record: Pega(Local) Reward
	System of record: Pega(Local) Evaluation Record
	System of record: Pega(Local) Audit Log
	System of record: Pega(Local) File Archive
	System of record: Pega(Local) Class Membership
	System of record: Pega(Local) Teacher Allocation
	System of record: Pega(Local) Subject Enrollment
	System of record: AI Evaluation Service Assessment Question
	System of record: AI Evaluation Service Submission Evaluation
	System of record: College ERP and Univer... Institution Subscription

By utilizing this file, you agree that the file and the contents are confidential and are provided solely for use with Pegasystems applications and products. Information in this file may have been generated by artificial intelligence and should be validated by a human prior to use. This file is provided as is. ©2026 Pegasystems Inc.

©2026 Pegasystems Inc.